

PYTHON CHEAT SHEET

PAGE
1/3

1. BASICS

- Python is a high-level, interpreted, general-purpose programming language.
- Easy to learn and read (simple syntax).
- Everything in Python is an object.
- # is used for single line comment
- Use triple quotes `""" """` or `''' '''` for multi-line string / docstring.

```
Hello World
print("Hello, World!")
```

```
Variables
x = 10
name = "Aman"
is_valid = True
```

2. DATA TYPES

- int : -10, 0, 25
- float : 3.14, -0.01
- str : "hello"
- bool : True, False
- list : [1, 2, 3]
- tuple : (1, 2, 3)
- set : {1, 2, 3}
- dict : {"name": "Aman", "age": 25}
- NoneType : None

3. TYPE CONVERSION

- int() float() str()
- list() tuple() set()
- dict() bool()

Example:

```
x = "10"
int(x) # 10
float(x) # 10.0
str(100) # "100"
```

4. OPERATORS

Arithmetic

+ - * / // % **

Comparison

== != > < >= <=

Logical

and or not

Assignment

= += -= *= /= %= //= **=

Bitwise

& | ^ ~ << >>

Membership

in not in

Identity

is is not

5. INPUT/OUTPUT

Input

```
name = input("Enter name: ")
```

Output

```
print("Hello", name)
print(f"Hello {name}")
```

Formatting

```
age = 25
print("Age: {}".format(age))
print(f"Age: {age}")
```

6. CONTROL FLOW

If - Elif - Else

```
if condition:
    pass
elif condition:
    pass
else:
    pass
```

For Loop

```
for i in range(5):
    print(i)
for item in list:
    print(item)
```

While Loop

```
while condition:
    pass
```

range(stop) → 0 to stop-1
range(start, stop) → start to stop-1
range(start, stop, step)
step can be +ve or -ve

7. STRINGS

Common Operations

- s[] # indexing
- s[start : end] # slicing
- len(s) # length
- s.lower() # to lowercase
- s.upper() # to uppercase
- s.strip() # remove spaces
- s.replace(old, new)
- s.split(delimiter)
- 'sub' in s # membership

String Methods

- capitalize()
- title()
- count(sub)
- find(sub)
- index(sub)
- startswith(x)
- endswith(x)
- isalpha()
- isdigit()
- isalnum()
- isspace()
- join(iterable)
- center(width)
- ljust() / rjust()

String Formatting (f-strings)

```
name = "Aman"
age = 25
print(f"My name is {name}
and age is {age}")
```

Strings are immutable,
i.e., cannot be changed.

PYTHON CHEAT SHEET

PAGE
2/3

8. LISTS (Mutable)

• Declaration

```
lst = [1, 2, 3]
```

• Indexing / Slicing

```
lst[0] # 1
```

```
lst[1:3] # [2, 3]
```

• Update / Change

```
lst[1] = 10
```

• Methods

- `append(x)`
- `extend(iterable)`
- `insert(i, x)`
- `remove(x)`
- `pop([i])`
- `clear()`
- `index(x)`
- `count(x)`
- `sort()`
- `reverse()`

Example

```
lst = [1, 2, 3]
```

```
lst.append(4)
```

```
lst → [1, 2, 3, 4]
```

9. TUPLES (Immutable)

• Declaration

```
t = (1, 2, 3)
```

• Indexing / Slicing

```
t[0] # 1
```

```
t[1:3] # (2, 3)
```

• Methods

- `count(x)`
- `index(x)`

Tuples are immutable
i.e., cannot be changed
after creation.

10. SETS (Unordered, Unique)

• Declaration

```
s = {1, 2, 3}
```

• Add / Remove

- `add(x)`
- `update(iterable)`
- `remove(x)`
- `discard(x)`
- `pop()`
- `clear()`

• Set Operations

- `union` (`s1 | s2`)
- `intersection` (`s1 & s2`)
- `difference` (`s1 - s2`)
- `symmetric diff` (`s1 ^ s2`)

11. DICTIONARY (Key-Value)

• Declaration

```
d = {"a": 1, "b": 2}
```

• Access / Update

```
d["a"] # 1
```

```
d["c"] = 3
```

• Methods

- `keys()`
- `values()`
- `items()`
- `get(key, default)`
- `pop(key, default)`
- `popitem()`
- `update(other_dict)`
- `clear()`
- `fromkeys(seq, value)`

12. LOOPS

For Loop

```
for i in range(5):  
    print(i)
```

```
for item in iterable:  
    print(item)
```

While Loop

```
while condition:  
    statements
```

```
break → exit loop  
continue → next iteration  
pass → do nothing
```

13. CONDITIONALS

if / elif / else

```
if condition:  
    statements  
elif another_condition:  
    statements  
else:  
    statements
```

Truthy Values

True, non-empty sequence,
non-zero numbers, etc.

Falsy Values

False, None, 0, "", [], (), {}

14. COMPREHENSIONS

List Comprehension

```
squares = [x*x for x in range(5)]
```

Dict Comprehension

```
d = {x: x*x for x in range(5)}
```

Set Comprehension

```
s = {x for x in range(5) if x%2==0}
```

15. FUNCTIONS

• Define Function

```
def add(a, b):  
    return a + b
```

• Default Arguments

```
def greet(name, msg="Hi"):  
    return msg + ", " + name
```

• *args and **kwargs

```
def fun(*args, **kwargs):  
    pass
```

Example

```
add(2, 3) # 5  
greet("Aman") # Hi, Aman  
fun(1, 2, 3, a=10, b=20)
```

16. LAMBDA, MAP, FILTER, REDUCE

Lambda (anonymous function)

```
square = lambda x: x*x  
print(square(5)) # 25
```

Map

```
nums = [1, 2, 3]  
sq = list(map(lambda x: x*x, nums))
```

Filter

```
evens = list(filter(lambda x: x%2==0,  
                    nums))
```

Reduce (from functools)

```
from functools import reduce  
from operator import add  
total = reduce(add, nums) # 6
```

17. MODULES & PACKAGES

• Import Module

```
import math  
from math import sqrt
```

• Install Package

```
pip install package_name
```

• Create Module

```
# mymodule.py  
def hello():  
    print("Hello")
```

Example

```
import math  
print(math.pi)  
print(sqrt(16))
```

A module is a .py file
A package is a directory
with `__init__.py` file.

PYTHON CHEAT SHEET

PAGE
3/3

18. OOPs (Object Oriented Programming)

Concepts

- Class & Object
- Constructor (`--init--`)
- Instance Variables
- Class Variables
- Inheritance
- Polymorphism
- Encapsulation
- Abstraction

Example

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age
    def greet(self):
        return f"Hi, I am {self.name}"

p = Person("Aman", 25)
print(p.greet())
```

19. EXCEPTIONS

Keywords

- try
- except
- else
- finally
- raise
- assert

Example

```
try:
    x = 10 / 0
except ZeroDivisionError as e:
    print("Error:", e)
else:
    print("No Error")
finally:
    print("Always Executed")
```

Custom Exception

```
raise ValueError("Invalid Value")
```

20. FILE HANDLING

Modes

- r - read
- w - write (overwrites)
- a - append
- x - create
- b - binary
- t - text (default)
- + - read & write

Example

```
with open('file.txt', 'r') as f:
    data = f.read()

with open('file.txt', 'w') as f:
    f.write("Hello")
```

Always close the file.
Better: use 'with' statement.

21. ITERATORS & GENERATORS

Iterator

- Object with `--iter--()` and `--next--()`
- `next()` raises `StopIteration` when no items left.

Example

```
Iterator
it = iter([1, 2, 3])
print(next(it)) # 1
print(next(it)) # 2
```

Generator

- Function with 'yield' returns generator object.
- Memory efficient (lazy evaluation).

Generator

```
def count():
    n = 1
    while True:
        yield n
        n += 1

c = count()
print(next(c)) # 1
```

22. COMMON BUILT-IN FUNCTIONS

- | | |
|--|---|
| • <code>abs(x)</code> - absolute value | • <code>filter(func, iter)</code> - filter items |
| • <code>all(iter)</code> - all True? | • <code>help(obj)</code> - help info |
| • <code>any(iter)</code> - any True? | • <code>id(obj)</code> - memory id |
| • <code>bin(x)</code> - binary | • <code>input()</code> - take input |
| • <code>bool(x)</code> - boolean | • <code>isinstance(obj, type)</code> - check type |
| • <code>chr(x)</code> - char from int | • <code>len(obj)</code> - length |
| • <code>dir(obj)</code> - attributes/methods | • <code>max(iter)</code> - maximum |
| • <code>enumerate(iter)</code> - with index | • <code>min(iter)</code> - minimum |

- | |
|---|
| • <code>map(func, iter)</code> - apply func |
| • <code>ord(ch)</code> - int from char |
| • <code>pow(x, y)</code> - x power y |
| • <code>range(start, stop, step)</code> |
| • <code>round(x, n)</code> - round off |
| • <code>sorted(iter)</code> - sorted list |
| • <code>sum(iter)</code> - sum of items |
| • <code>zip(a, b)</code> - pair elements |

23. USEFUL TIPS

- ✓ Use list/dict/set comprehensions.
- ✓ Prefer `enumerate` over `range(len())`.
- ✓ Use `zip()` to iterate over multiple iterables.
- ✓ Use 'in' for membership checks (fast).
- ✓ Profile your code (`timeit`, `cProfile`).
- ✓ Write clean, readable and modular code.



24. COMMON PATTERNS / TRICKS

- Swap two numbers: `a, b = b, a`
- Find duplicate in list: `[x for x in set(lst) if lst.count(x) > 1]`
- Flatten list: `[item for sub in matrix for item in sub]`
- Reverse string: `s[::-1]`
- Check palindrome: `s == s[::-1]`
- Count frequency: `{x: lst.count(x) for x in set(lst)}`
- Merge dicts (Python 3.9+): `d1 | d2`
- Remove duplicates: `list(dict.fromkeys(lst))`
- Find missing number (1..n): `n*(n+1)//2 - sum(lst)`

25. COMMON ALGORITHMS (Quick Reference)

- | | |
|--------------------|--|
| • Factorial | • Sorting (Bubble, Selection, Insertion, Merge, Quick) |
| • Fibonacci Series | • Two Pointers |
| • Prime Check | • Sliding Window |
| • GCD / LCM | • Recursion / Backtracking |
| • Binary Search | • Dynamic Programming (Basics) |
| • Linear Search | |

26. PYTHON VERSION FEATURES (Quick Glance)

Version	Key Features
3.6	f-strings, underscores in numbers
3.7	dataclasses, dict order guaranteed
3.8	walrus operator (<code>:=</code>), positional-only args
3.9	dict merge (<code> </code>), updated modules
3.10	match-case, better error messages
3.11	performance improvements
3.12	further speed & memory optimizations

27. KEYWORDS (Reserved Words)

False	class	finally	is	return
None	continue	for	lambda	try
True	def	from	nonlocal	while
and	del	global	not	with
as	elif	if	or	yield
assert	else	import	pass	
break	except	in	raise	

Keywords cannot be used as identifiers.